当前阶段进展汇报

一、项目目标

当前主流 LLM 辅助 fuzzing 框架虽然已经能够自动生成 fuzz driver 并跑出 crash，但 crash 出来以后仍存在几个明显问题：

- 高误判率 | LLM 倾向于把所有 crash 都判定为"目标库的 API bug",在 c-ares 项目的实验中，原始方法把 92.3% 的 crash 判定为 API bug，但实际上许多是 driver 自身的问题
- 定位模糊 | 原始方法只知道哪个 fuzz driver 触发了 crash，不能告诉你 crash 出现在目标库源码的哪个文件、哪一行
- 缺乏运行时上下文 | 没有自动化的 PoC 复现验证机制，也没有收集 crash 时刻的变量值、寄存器状态等运行时信息
- 重复分析浪费 | 同一个 root cause 可能被多个不同的 fuzz driver 反复触发，每次都消耗 LLM 调用（token 费用 + 等待时间）
- 输出内容单薄 | 分析结果只有一个判读结果 `is_api_bug` + 一个 `crash_category` 字符串 + 自由文本描述，缺乏 root cause 类型、severity 严重度、data flow 数据流、fix suggestion 修复建议等结构化字段

本工作的核心目标，在现有 `CKGFuzzer` 基础上，补齐其在 crash 后处理环节上的关键短板

二、当前方案概述

目前已经形成了一个六阶段的 crash 分析流水线：

1. `Sanitizer` 结构化解析
   把原始 crash 日志转成统一结构，提取
   `bug_type`（错误类型） 、 `stack_frames`（栈帧列表） 、 `scariness_score(严重性评分)` 、崩溃地址等字段
   这一阶段的目标是把文本日志变成统一字段，为后续规则判断、签名构建和 LLM 输入提供基础
2. 规则粗筛
   基于 bug 类型、栈帧位置、driver/库代码分布等特征，先对 crash 做低成本筛选，输出
   `noise / likely_driver_bug / likely_api_bug / needs_review`
   该阶段优先拦截明显低价值样本（如 driver 明显越界、无关模块崩溃），从而把后续高成本分析预算集中在可疑 API crash 上

3. 代码定位与调用链补全
   从栈帧中定位目标库源码中的可疑位置，提取文件、函数、行号以及局部代码上下文
   在这一阶段会尽量恢复“触发路径 -> 崩溃点 -> 上游调用关系”，避免模型只看到单个崩溃点而缺乏语义链路
4. PoC 复现 + GDB 运行时采集
   尝试对 crash 做自动复现，并利用 GDB 测试工具补充崩溃时的帧信息和运行时上下文
   若复现成功，会追加寄存器/关键变量快照等动态证据，用于辅助区分“真实 API 缺陷”与“偶发执行噪声”
5. 多级签名去重
   基于精确签名、栈签名、模糊签名三层策略，对同源 crash 做聚类，只保留代表样本进入深度分析
   该阶段的核心是控制重复分析成本：同一 root cause 由多个输入触发时，仅保留代表样本交给后续模型判定
6. 上下文增强 LLM 分析
   将前面阶段收集到的结构化证据统一打包，再交给 LLM 输出结构化分析结果
   输出包括漏洞类型判断、置信度、关键证据引用等，可用于u后续人工审查等后续工作

该方案的整体思想是：

- 前置阶段先减少不值得分析的样本
- 后置阶段再对少量高价值 crash 做更精确的定位与解释
- 最终同时提升准确性与分析效率

三、消融实验设计

为了拆分各模块的贡献，目前采用五种变体做对照：

变体	规则粗筛	源码定位	GDB 运行时上下文	去重	LLM 分析	主要含义
V0	-	-	-	-	yes	原始基线，仅把 sanitizer 日志交给 LLM
V1	yes	-	-	-	yes	只看规则粗筛的贡献
V2	-	yes	yes	-	yes	只看上下文增强的贡献
V3	yes	-	-	yes	yes	看效率链路：粗筛 + 去重
V4	yes	yes	yes	yes	yes	完整方案

这个设计重点回答 3 个问题：

- 规则粗筛单独是否有效
- 上下文增强单独是否有效
- 两类能力结合后是否具有互补增益

四、当前实验进度

4.1 项目进展

- `c-ares` 、 `cJSON` ： 已完成完整实验
- `其余项目` ： 待测

4.2 各项目数据

c-ares：

- 共获得 `45` 个 crash
- 来自 `10` 个 fuzz driver
- 当前 silver label 统计： `True = 4` ， `False = 41`

cJSON：

- 共获得 `30` 个 crash
- 来自 `19` 个 fuzz driver
- 当前 silver label 统计： `True = 2` ， `False = 28`

合计：

- 共 `75` 个 crash， `True = 6` ， `False = 69`

4.3 标签定义说明

现阶段 `True/False` 的含义并不是“绝对真值”，而是基于当前证据做出的高置信度判断： `True` 表示“极大概率是 API bug”， `False` 表示“极大概率不是 API bug（更可能是 driver bug、noise 或非目标库问题）”

4.4 消融结果（c-ares + cJSON）

Variant	Description	Precision	Recall	F1	LLM Calls	TP	FP	TN	FN
V0	Baseline (LLM only)	0.143	0.667	0.235	75	4	24	45	2
V1	Triage-Only	0.300	1.000	0.462	49	6	14	55	0
V2	Context-Only	0.174	0.667	0.276	75	4	19	50	2
V3	Triage + Dedup	0.429	1.000	0.600	14	6	8	61	0
V4	CrashSage Full	0.750	1.000	0.857	14	6	2	67	0

五、当前阶段结论

基于 `c-ares + cJSON` 两个项目的合计结果，目前可以得到以下几个明确结论：

5.1 规则粗筛本身有效

`V1` 相比 `V0`：

- F1 从 `0.235` 提升到 `0.462`（+0.227）
- Recall 从 `0.667` 提升到 `1.000`
- LLM 调用次数从 `75` 降到 `49`（少 26 次）

说明：即便不引入额外代码上下文，仅做前置规则 triage，也已经能够同时改善准确率和成本

5.2 上下文增强主要提升质量

`V2` 相比 `V0`：

- F1 从 `0.235` 提升到 `0.276`（+0.041）
- LLM 调用次数保持 `75` 不变

说明：源码定位和运行时上下文主要作用在“判得更准”，但本身不减少分析量

5.3 去重对效率提升非常显著

`V3` 相比 `V1`：

- F1 从 `0.462` 提升到 `0.600`（+0.138）
- LLM 调用次数从 `49` 进一步降到 `14`（少 35 次）

说明：在经过初筛后，再做同源 crash 聚合，可以显著减少重复分析，是效率提升的关键模块

5.4 完整方案取得当前最佳结果

`V4` 相比 `V3`：

- LLM 调用次数保持 `14` 不变
- F1 从 `0.600` 提升到 `0.857`（+0.257）
- Precision 从 `0.429` 提升到 `0.750`（FP 从 8 降到 2）
- Recall 保持 `1.000` 不变

说明：在相同调用预算下，进一步补充源码与运行时上下文，主要帮助消除误报

5.5 两类组件互补

V4 的 F1= `0.857` 优于前三个变体中的最佳者 V3（F1= `0.600` ），且 LLM 调用次数不增加（均为 `14` ），说明效率维度（粗筛+去重）与质量维度（源码定位+运行时上下文）是互补关系

一句话概括就是：

V0 到 V4，LLM 调用从 75 次降至 14 次（节约 81.3%），F1 从 0.235 提升至 0.857，Recall 达到 1.000

六、现阶段存在的问题

poc复现效果一般

当前阶段 4 的自动复现与 GDB 采集并不稳定，部分 crash 的 `repro_rate`（复现率） 较低，或者即使复现成功，也无法稳定采集到足够有价值的变量/寄存器信息

可能原因包括：

- ASan 二进制被 coverage 版本覆盖
- 部分 crash 本身具有非确定性

- 容器环境下 GDB 兼容性有限

这意味着：

V4 相比 V3 的收益，当前更多来自源码定位与上下文补全，而不是 GDB 动态证据